

21CSE451T

PATTERN RECOGNITION TECHNIQUES

UNIT-2

INTRODUCTION TO PATTERN RECOGNITION SYSTEMS

- **Parameter Estimation Methods**
 - **Maximum-Likelihood Estimation**
 - **Bayesian Estimation**
- **Bayesian Parameter Estimation: Gaussian Case**
- **Bayesian Parameter Estimation: General Theory**
- **Problems of Dimensionality**
- **Component Analysis and Discriminants**
- **Expectation-Maximization**
- **Hidden Markov Model**

PARAMETER ESTIMATION METHODS

- In typical supervised pattern classification problems, the estimation of the prior probabilities presents no serious difficulties. However, estimation of the class-conditional densities is quite another matter.
- The number of available samples always seems too small, and serious problems arise when the dimensionality of the feature vector x is large.
- If we know the number of parameters in advance and our general knowledge about the problem permits us to parameterize the conditional densities, then the severity of these problems can be reduced significantly.
- Suppose assume that $p(x|\omega_i)$ is a normal density with **mean μ_i , and covariance matrix Σ_i** , although we do not know the exact values of these quantities.
- This knowledge simplifies the problem from one of estimating an unknown *function* $p(x|\omega_i)$ to one of **estimating the parameters μ_i and Σ_i** .
- Two common approaches for parameter estimation: ***Maximum Likelihood Estimation and Bayesian Estimation***. Although the results obtained from these are almost same.
 - **Maximum-likelihood** view the parameters as quantities whose values are fixed but unknown. The best estimate of their value is defined to be the one that maximizes the probability of obtaining the samples actually observed.
 - **Bayesian methods** view the parameters as random variables having some known prior distribution. Observation of the samples converts this to a posterior density, thereby revising our opinion about the true values of the parameters. In

the Bayesian case, we shall see that a typical effect of observing additional samples is to sharpen the *a posteriori* density function, causing it to peak near the true values of the parameters. This phenomenon is known as **Bayesian learning**.

MAXIMUM-LIKELIHOOD ESTIMATION

Maximum-likelihood estimation methods have a number of attractive attributes. First, they nearly always have good convergence properties as the number of training samples increases. Furthermore, maximum-likelihood estimation often can be simpler than alternative methods, such as Bayesian techniques.

The General Principle

- Suppose that we separate a collection of samples according to class, so that we have c data sets, $D_1, \dots, D_c$, with the samples in D_j having been drawn independently according to the probability law $p(x|w_j)$. We say such samples are *i.i.d.*—independent and identically distributed random variables. We assume that $p(x|w_j)$ has a known parametric form, and is therefore determined uniquely by the value of a parameter vector θ_j .
- For example, we might have $p(x|w_j) \sim N(\mu_j, \Sigma_j)$ where θ_j consists of the components of μ_j , and Σ_j . To show the dependence of $p(x|w_j)$ on θ_j explicitly, we write $p(x|w_j)$ as $p(x|w_j, \theta_j)$. Our problem is to use the information provided by the training samples to obtain good estimates for the unknown parameter vectors $\theta_1, \dots, \theta_c$ associated with each category.
- To simplify treatment of this problem, we shall assume that samples in D_i give no information about θ_j if $i \neq j$; that is, we shall assume that the parameters for the different classes are functionally independent. With this assumption we thus have c separate problems of the following form:
- Use a set D of training samples drawn independently from the probability density $p(x|\theta)$ to estimate the unknown parameter vector θ .
- Suppose that D contains n samples, $x_1, \dots, x_n$. Then, because the samples were drawn independently, we have:

$$p(D|\theta) = \prod_{k=1}^n p(x_k|\theta)$$

- $p(D|\theta)$ is called the *likelihood* of θ with respect to the set of samples. The *maximum-likelihood estimate* of θ is, by definition, the value $\hat{\theta}$ that maximizes $p(D|\theta)$.

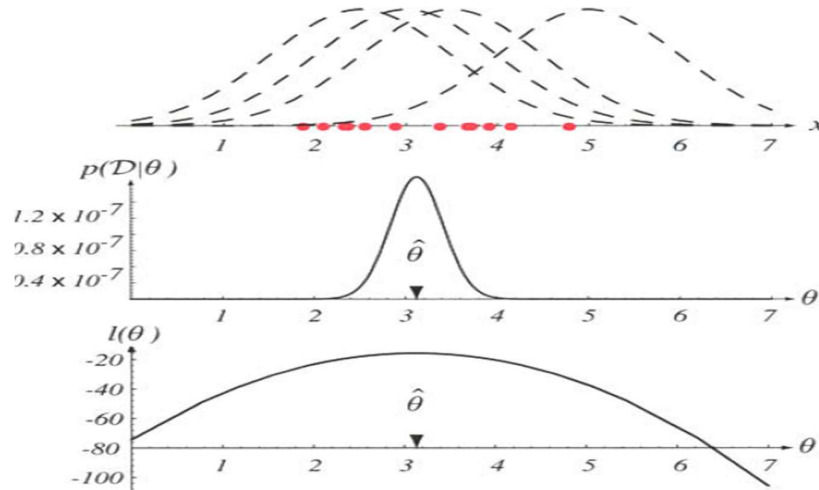


Figure: The top graph shows several training points in one dimension, known or assumed to be drawn from a Gaussian of a particular variance, but unknown mean. Four of the infinite number of candidate source distributions are shown in dashed lines. The middle figure shows the likelihood $p(D|\theta)$ as a function of the mean. If we had a very large number of training points, this likelihood would be very narrow. The value that maximizes the likelihood is marked $\hat{\theta}$; it also maximizes the logarithm of the likelihood—that is, the log-likelihood $l(\theta)$, shown at the bottom. Note that even though they look similar, the likelihood $p(D|\theta)$ is shown as a function of θ whereas the conditional density $p(x|\theta)$ is shown as a function of x . Furthermore, as a function of θ , the likelihood $p(D|\theta)$ is not a probability density function and its area has no significance.

If the number of parameters to be estimated is p , then we let θ denote the p -component vector $\theta = (\theta_1, \dots, \theta_p)^t$ and let the gradient operator:

$$\nabla_{\theta} \equiv \begin{bmatrix} \frac{\partial}{\partial \theta_1} \\ \vdots \\ \frac{\partial}{\partial \theta_p} \end{bmatrix}.$$

We define $l(\theta)$ as the log-likelihood function*

$$l(\theta) \equiv \ln p(D|\theta).$$

We can then write our solution formally as the argument θ that maximizes the likelihood, that is,

$$\hat{\theta} = \arg \max_{\theta} l(\theta),$$

where the dependence on the data set $\mathcal{D}$ is implicit. Thus we have from Eq. 1

$$l(\theta) = \sum_{k=1}^n \ln p(\mathbf{x}_k|\theta)$$

and

$$\nabla_{\theta} l = \sum_{k=1}^n \nabla_{\theta} \ln p(\mathbf{x}_k|\theta).$$

Thus, a set of necessary conditions for the maximum-likelihood estimate for θ can be obtained from the set of p equations

$$\nabla_{\theta} l = \mathbf{0}.$$

Maximum a posteriori or MAP estimators—find the value of θ that maximizes $l(\theta) + \ln p(\theta)$, where $p(\theta)$ describes the prior probability of different parameter values.

1. The Gaussian Case: Unknown μ

Suppose that the samples are drawn from a multivariate normal population with mean μ and covariance matrix Σ . Find:

$$\ln p(\mathbf{x}_k | \mu) = -\frac{1}{2} \ln [(2\pi)^d |\Sigma|] - \frac{1}{2} (\mathbf{x}_k - \mu)' \Sigma^{-1} (\mathbf{x}_k - \mu)$$

and

$$\nabla_{\mu} \ln p(\mathbf{x}_k | \mu) = \Sigma^{-1} (\mathbf{x}_k - \mu).$$

$$\sum_{k=1}^n \Sigma^{-1} (\mathbf{x}_k - \hat{\mu}) = \mathbf{0}.$$

Multiplying by Σ and rearranging, we obtain

$$\hat{\mu} = \frac{1}{n} \sum_{k=1}^n \mathbf{x}_k.$$

2. The Gaussian Case: Unknown μ , and Σ

In the more general (and more typical) multivariate normal case, neither the mean μ nor the covariance matrix Σ is known. Thus, these unknown parameters constitute the components of the parameter vector Θ . Consider first the univariate case with $\Theta_1 = \mu$ and $\Theta_2 = \sigma^2$. Here the log-likelihood of a single point is:

$$\ln p(x_k | \theta) = -\frac{1}{2} \ln 2\pi\theta_2 - \frac{1}{2\theta_2} (x_k - \theta_1)^2$$

and its derivative is

$$\nabla_{\theta} l = \nabla_{\theta} \ln p(x_k | \theta) = \begin{bmatrix} \frac{1}{\theta_2} (x_k - \theta_1) \\ -\frac{1}{2\theta_2} + \frac{(x_k - \theta_1)^2}{2\theta_2^2} \end{bmatrix}$$

full log-likelihood leads to the conditions

$$\sum_{k=1}^n \frac{1}{\hat{\theta}_2} (x_k - \hat{\theta}_1) = 0$$

$$-\sum_{k=1}^n \frac{1}{\hat{\theta}_2} + \sum_{k=1}^n \frac{(x_k - \hat{\theta}_1)^2}{\hat{\theta}_2^2} = 0,$$

$$\hat{\mu} = \frac{1}{n} \sum_{k=1}^n x_k$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{k=1}^n (x_k - \hat{\mu})^2$$

2) Bias

The maximum-likelihood estimate for the variance σ^2 is *biased*; that is, the expected value over all data sets of size n of the sample variance is not equal to the true variance:

$$\mathcal{E} \left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right] = \frac{n-1}{n} \sigma^2 \neq \sigma^2.$$

An elementary *unbiased* estimator for Σ is given by

$$\mathbf{C} = \frac{1}{n-1} \sum_{k=1}^n (\mathbf{x}_k - \hat{\boldsymbol{\mu}})(\mathbf{x}_k - \hat{\boldsymbol{\mu}})^t$$

where $\mathbf{C}$ is the so-called *sample covariance matrix*.

BAYESIAN ESTIMATION

Maximum-likelihood methods we view the true parameter vector we seek, Θ , to be fixed, in Bayesian learning we consider Θ to be a random variable, and training data allow us to convert a distribution on this variable into a posterior probability density.

1. **The Class-Conditional Densities:** If we again let D denote the set of samples, then we can emphasize the role of the samples by saying that our goal is to compute the posterior probabilities $P(\omega_i | \mathbf{x}, D)$. From these probabilities we can obtain the Bayes classifier.

Given the sample D , Bayes formula then becomes:

$$P(\omega_i | \mathbf{x}, D) = \frac{p(\mathbf{x} | \omega_i, D) P(\omega_i | D)}{\sum_{j=1}^c p(\mathbf{x} | \omega_j, D) P(\omega_j | D)}.$$

Assume that the true values of the prior probabilities are known or obtainable from a trivial calculation; thus we substitute $P(\omega_i) = P(\omega_i | \mathbf{x}, D)$.

$$P(\omega_i | \mathbf{x}, D) = \frac{p(\mathbf{x} | \omega_i, D_i) P(\omega_i)}{\sum_{j=1}^c p(\mathbf{x} | \omega_j, D_j) P(\omega_j)}$$

2. The Parameter Distribution

Basic goal is to compute $p(\mathbf{x} | D)$, which is as close as we can come to obtaining the unknown $p(\mathbf{x})$. We do this by integrating the joint density $p(\mathbf{x}, \boldsymbol{\theta} | D)$ over $\boldsymbol{\theta}$. That is,

$$p(\mathbf{x} | D) = \int p(\mathbf{x}, \boldsymbol{\theta} | D) d\boldsymbol{\theta}.$$

The above equation can be written as:

$$p(\mathbf{x} | D) = \int p(\mathbf{x} | \boldsymbol{\theta}) p(\boldsymbol{\theta} | D) d\boldsymbol{\theta}.$$

BAYESIAN PARAMETER ESTIMATION: GAUSSIAN CASE

Use Bayesian estimation techniques to calculate the *a posteriori* density $p(\boldsymbol{\theta} | D)$ and the desired probability density $p(\mathbf{x} | D)$ for the case where $p(\mathbf{x} | \boldsymbol{\mu}) \sim N(\boldsymbol{\mu} | \Sigma)$.

1. The Univariate Case: $p(\boldsymbol{\mu} | D)$

Consider the case where $\boldsymbol{\mu}$ is the only unknown parameter. For simplicity we treat first the univariate case.

$$p(x|\mu) \sim N(\mu, \sigma^2)$$

Gaussian Form:

$$p(\mu|\mathcal{D}) = \frac{1}{\sqrt{2\pi}\sigma_n} \exp\left[-\frac{1}{2}\left(\frac{\mu - \mu_n}{\sigma_n}\right)^2\right]$$

[identifying coefficients in this way yields

$$\frac{1}{\sigma_n^2} = \frac{n}{\sigma^2} + \frac{1}{\sigma_0^2}$$

and

$$\frac{\mu_n}{\sigma_n^2} = \frac{n}{\sigma^2} \hat{\mu}_n + \frac{\mu_0}{\sigma_0^2},$$

where $\hat{\mu}_n$ is the sample mean

$$\hat{\mu}_n = \frac{1}{n} \sum_{k=1}^n x_k.$$

$$\sigma_n^2 = \frac{\sigma_0^2 \sigma^2}{n\sigma_0^2 + \sigma^2}.$$

2. The Univariate Case: $p(x|\mathcal{D})$

Having obtained the *a posteriori* density for the mean, $p(\mu|\mathcal{D})$, all that remains is to obtain the "class-conditional" density for $p(x|\mathcal{D})$.

$$\begin{aligned} p(x|\mathcal{D}) &= \int p(x|\mu) p(\mu|\mathcal{D}) d\mu \\ &= \int \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right] \frac{1}{\sqrt{2\pi}\sigma_n} \exp\left[-\frac{1}{2}\left(\frac{\mu-\mu_n}{\sigma_n}\right)^2\right] d\mu \\ &= \frac{1}{2\pi\sigma\sigma_n} \exp\left[-\frac{1}{2}\frac{(x-\mu_n)^2}{\sigma^2 + \sigma_n^2}\right] f(\sigma, \sigma_n), \end{aligned} \quad (36)$$

where

$$f(\sigma, \sigma_n) = \int \exp\left[-\frac{1}{2}\frac{\sigma^2 + \sigma_n^2}{\sigma^2\sigma_n^2}\left(\mu - \frac{\sigma_n^2 x + \sigma^2 \mu_n}{\sigma^2 + \sigma_n^2}\right)^2\right] d\mu.$$

That is, as a function of x , $p(x|\mathcal{D})$ is proportional to $\exp[-(1/2)(x-\mu_n)^2/(\sigma^2 + \sigma_n^2)]$ and hence $p(x|\mathcal{D})$ is normally distributed with mean μ_n and variance $\sigma^2 + \sigma_n^2$:

$$p(x|\mathcal{D}) \sim N(\mu_n, \sigma^2 + \sigma_n^2). \quad (37)$$

3. The Multivariate Case

The treatment of the multivariate case in which Σ is known but μ is not is a direct generalization of the univariate case.

$$p(\mathbf{x}|\boldsymbol{\mu}) \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad \text{and} \quad p(\boldsymbol{\mu}) \sim N(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0),$$

BAYESIAN PARAMETER ESTIMATION: GENERAL THEORY

The Bayesian approach can be used to obtain the desired density $p(x|D)$ in a special case—the multivariate Gaussian. This approach can be generalized to apply to any situation in which the unknown density can be parameterized.

The basic assumptions are summarized as follows:

- The form of the density $p(x|\theta)$ is assumed to be known, but the value of the parameter vector θ is not known exactly.
- Our initial knowledge about θ is assumed to be contained in a known prior density $p(\theta)$.
- The rest of our knowledge about θ is contained in a set D of n samples $x_1, \dots, x_n$ drawn independently according to the unknown probability density $p(x)$.

The basic problem is to compute the posterior density $p(\theta|D)$:

$$p(\mathbf{x}|D) = \int p(\mathbf{x}|\theta)p(\theta|D) d\theta$$

By Bayes formula we have

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{\int p(D|\theta)p(\theta) d\theta},$$

and by the independence assumption

$$p(D|\theta) = \prod_{k=1}^n p(\mathbf{x}_k|\theta).$$

To indicate explicitly the number of samples in a set for a single category, we shall write $D^n = \{x_1, x_2, \dots, x_n\}$. If $n > 1$,

$$p(D^n|\theta) = p(\mathbf{x}_n|\theta)p(D^{n-1}|\theta)$$

Using Bayes formula, we see that the posterior density satisfies the recursion relation:

$$p(\theta|D^n) = \frac{p(\mathbf{x}_n|\theta)p(\theta|D^{n-1})}{\int p(\mathbf{x}_n|\theta)p(\theta|D^{n-1}) d\theta}$$

When Do Maximum-Likelihood and Bayes Methods Differ?

There are several criteria that will influence our choice.

- One is computational complexity, and here maximum-likelihood methods are often to be preferred because they require merely differential calculus techniques or gradient search for θ^* , rather than a possibly complex multidimensional integration needed in Bayesian estimation.
- This leads to another consideration: interpretability. In many cases the maximum-likelihood solution will be easier to interpret and understand because it returns the single best model from the set the designer provided. In contrast, Bayesian methods give a weighted average of models (parameters), often leading to solutions more complicated and harder to understand than those provided by the designer. The Bayesian approach reflects the remaining uncertainty in the possible models.
- Another consideration is our confidence in the prior information, such as in the *form* of the underlying distribution $p(x|\theta)$. A maximum-likelihood solution $p(x|\theta^*)$ must of course be of the assumed parametric form; not so for the Bayesian solution.

- When $p(\theta|D)$ is broad, or asymmetric around θ^* , the methods are quite likely to yield $p(x|\theta)$ distributions that differ from one another. Such a strong asymmetry: Bayes methods would exploit such information, whereas maximum-likelihood methods would not.
- When designing a classifier by either of these methods, we determine the posterior densities for each category and also classify a test point by the maximum posterior. (If there are costs, summarized in a cost matrix, these can be incorporated as well.)

There are three sources of classification error in our final system:

- **Bayes or Indistinguishability Error:** The error due to overlapping densities $p(x|\omega_i)$ for different values of i . This error is an inherent property of the problem and can never be eliminated.
- **Model Error:** The error due to having an incorrect model. This error can only be eliminated if the designer specifies a model that includes the true model which generated the data. Designers generally choose the model based on knowledge of the problem domain rather than on the subsequent estimation method, and thus the model error in maximum-likelihood and Bayes methods rarely differ.
- **Estimation Error:** The error arising from the fact that the parameter's are estimated from a finite sample. This error can best be reduced by increasing the training data.

Noninformative Priors and Invariance

In a Bayesian framework we can have a "noninformative" prior over a parameter for a single category's distribution. Suppose for instance that we are using Bayesian methods to infer from data some position and scale parameters, which we denote μ and σ , which might be the mean and standard deviation of a Gaussian, or the position and width of a triangle distribution.

Gibbs Algorithm

To pick a parameter vector θ according to $p(\theta|D)$ and use the single value as if it were the true value. Given weak assumptions, this so-called *Gibbs algorithm* gives a misclassification error that is at most twice the expected error of the Bayes optimal classifier.

PROBLEMS OF DIMENSIONALITY

In practical multicategory applications, it is not at all unusual to encounter problems involving fifty or a hundred features, particularly if the features are binary valued.

1. Accuracy, Dimension, and Training Sample Size

If the features are statistically independent, there are some theoretical results that suggest the possibility of excellent performance. For example, consider the two-class multivariate normal case with the same covariance, i.e., where $p(x|\omega_j) \sim N(\mu_j, \Sigma)$, $j = 1, 2$. If the prior probabilities are equal, then it is not hard to show that the Bayes error rate is given by:

$$P(e) = \frac{1}{\sqrt{2\pi}} \int_{r/2}^{\infty} e^{-u^2/2} du.$$

where r^2 is the squared Mahalanobis distance

$$r^2 = (\mu_1 - \mu_2)' \Sigma^{-1} (\mu_1 - \mu_2)$$

Thus, the probability of error decreases as r increases, approaching zero as r approaches infinity. In the conditionally independent case, $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$:

$$r^2 = \sum_{i=1}^d \left(\frac{\mu_{i1} - \mu_{i2}}{\sigma_i} \right)^2$$

2. Computational Complexity

First, we will need to understand the notion of the *order* of a function $f(x)$: we say that the $f(x)$ is "of the order of $h(x)$ "—written $f(x) = O(h(x))$ and generally read "big oh of $h(x)$ "—if there exist constants c and x_0 such that $|f(x)| \leq c|h(x)|$ for all $x > x_0$. This means simply that for sufficiently large x , an upper bound on the function grows no worse than $h(x)$. For instance, suppose $f(x) = a_0 + a_1 x + a_2 x^2$; in that case we have $f(x) = O(x^2)$ because for sufficiently large x the constant, linear, and quadratic terms can be "overcome" by proper choice of c and x_0 . The generalization to functions of two or more variables is straightforward. It should be clear that by the definition above, the big oh order of a function is not unique. For instance, we can describe our particular $f(x)$ as being $O(x^2)$, $O(x^3)$, $O(x^4)$ or $O(x^2 \ln x)$.

3. Overfitting

One possibility is to reduce the dimensionality, either by redesigning the feature extractor, by selecting an appropriate subset of the existing features, or by combining the existing features in some way.

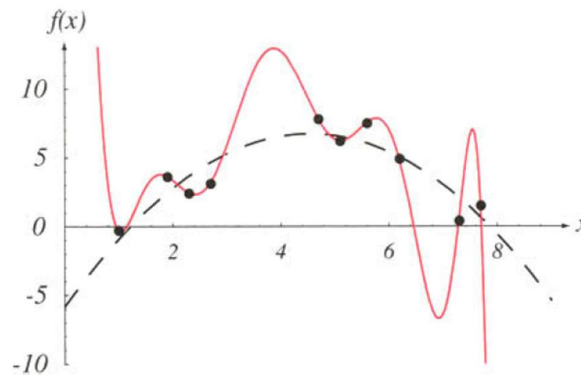


Figure: The "training data" (black dots) were selected from a quadratic function plus Gaussian noise, i.e., $f(x) = ax^2 + bx + c + \epsilon$ where $p(\epsilon) \sim N(0, \sigma^2)$. The 10th-degree polynomial shown fits the data perfectly, but we desire instead the second-order function $f(x)$, because it would lead to better predictions for new samples.

COMPONENT ANALYSIS AND DISCRIMINANTS

One approach to coping with the problem of excessive dimensionality is to reduce the dimensionality by combining features. Linear combinations are particularly attractive because they are simple to compute and analytically tractable. In effect, linear methods project the high-dimensional data onto a lower dimensional space.

There are two classical approaches to finding effective linear transformations.

1. **Principal Component Analysis or PCA**—seeks a projection that best represents the data in a least-squares sense.

2. **Fischer Linear Discriminant or FLD**- find the best linear combination of features that separates two or more classes of data
3. **Multiple Discriminant Analysis or MDA**—seeks a projection that best separates the data in a least-squares sense.

1. Principal Component Analysis (PCA) is a **dimensionality reduction technique** used to simplify complex datasets while preserving as much variability (information) as possible.

Purpose of PCA

- Reduce the number of features (variables) in a dataset while retaining the most important patterns.
- Remove redundancy caused by correlated variables.
- Help with visualization when data has many dimensions (e.g., reducing 50 features to 2 or 3).
- Speed up machine learning algorithms by working with fewer variables.

PCA **finds new axes** (called *principal components*) that:

- Are **linear combinations** of the original features.
- Capture the **maximum variance** in the data.
- Are **uncorrelated** with each other.

Steps of PCA

1. **Standardize the Data**
 - PCA is affected by scale, so data is often standardized to have mean 0 and variance 1.
2. **Compute the Covariance Matrix**
 - Measures how variables vary together.
 - Example: If height and weight are correlated, their covariance will be high.
3. **Find Eigenvalues and Eigenvectors**
 - **Eigenvectors** → directions of new feature axes (principal components).
 - **Eigenvalues** → how much variance each principal component captures.
4. **Sort Components by Variance**
 - Keep the top k principal components that explain most of the variance.
5. **Transform the Data**
 - Project the original data onto the selected principal components.

Example

Imagine you have two features:

- **Height**
- **Weight**

Both are correlated. PCA will:

- Create **PC1** (largest variance direction – maybe “body size”).
- Create **PC2** (smaller variance direction – maybe “slenderness”).
- You can drop PC2 if it explains very little variance.

When to Use PCA

- Large datasets with many features.
- High correlation among variables.
- You need faster computation or better visualization.
- Preprocessing before machine learning to avoid overfitting.

2. Fischer Linear Discriminant or FLD

FLD is also known as **Linear Discriminant Analysis (LDA)** in the two-class case, but its *goal* is different from PCA.

Purpose of FLD

- **PCA**: Finds directions that capture maximum variance (unsupervised — ignores class labels).
- **FLD**: Finds a direction that *best separates two or more classes* (supervised — uses class labels).
- Main use: Dimensionality reduction *while maximizing class separability*.

FLD projects data onto a line so that:

- The **distance between class means** is as large as possible.
- The **variance within each class** is as small as possible.
- This is measured by the **Fisher criterion**:

$$J(w) = (m_1 - m_2)^2 / (s_1^2 + s_2^2)$$

where:

- m_1, m_2 are projected means of the two classes.
- s_1^2, s_2^2 are projected variances.

Steps for Two-Class FLD

1. **Compute class means** μ_1, μ_2 in the original space.
2. **Compute scatter within each class** (within-class scatter matrices S_1, S_2).
3. **Within-class scatter matrix**:
 $S_W = S_1 + S_2$
4. **Find projection vector**:
 $w = S_W^{-1}(\mu_1 - \mu_2)$
5. **Project data**:
 $y = w^T x$
6. **Classify**:
 - Choose a threshold (e.g., midpoint of projected means).
 - Points on one side → Class 1, other side → Class 2.

Example Intuition

Imagine two classes in 2D: red points and blue points.

- PCA might pick the longest variance direction — which could cut *through* the classes and not separate them well.
- FLD finds the line that best separates red and blue when projected into 1D.

3. Multiple Discriminant Analysis or MDA

Multiple Discriminant Analysis (MDA) is essentially the *multi-class* generalization of **Fisher's Linear Discriminant (FLD)**.

It's also commonly known as **Linear Discriminant Analysis (LDA)** in statistics and machine learning when there are more than two classes.

- FLD works for **two classes** — one projection vector w separates them.
- MDA works for **k classes** — it finds up to $k-1$ new axes (discriminant functions) that maximize *overall* class separability.
- It is **supervised dimensionality reduction** — reduces data from d -dimensions to at most $k-1$ dimensions while preserving class discrimination.

Core Idea

Instead of just maximizing the separation between **two** means, MDA:

- Maximizes the separation between **all class means**.
- Minimizes the variance **within** each class.

- Uses the **between-class scatter** S_B and **within-class scatter** S_W .

EXPECTATION-MAXIMIZATION

The Expectation-Maximization (EM) algorithm is an iterative method used to find the maximum likelihood or maximum a posteriori estimates of parameters in statistical models, particularly when dealing with hidden or incomplete data. It's frequently applied in machine learning, especially in situations where some data points are missing or unobserved (latent variables).

The **EM algorithm** is used to **find the best parameters** of a statistical model when:

- Some data is **missing, hidden, or latent**.
- The likelihood function is **hard to optimize directly**.

It's widely used for:

- **Clustering** (e.g., Gaussian Mixture Models).
- **Missing data imputation**.
- **Hidden Markov Models**.

EM alternates between:

1. **E-step (Expectation)** → Estimate missing/hidden variables given the current parameter guesses.
2. **M-step (Maximization)** → Re-estimate parameters by maximizing the expected log-likelihood from the E-step.

This process repeats until convergence.

Why Not Optimize Directly?

- If we knew the missing/hidden values, parameter estimation would be easy (just like normal maximum likelihood estimation).
- If we knew the parameters, inferring hidden values would be easy.
- But when *both* are unknown, EM solves it **iteratively**.

Step-by-Step (Generic EM)

Given:

- Observed data X
- Hidden variables Z
- Model parameters θ

Step 1 — Initialize Parameters: Choose initial guess for parameters $\theta^{(0)}$.

Step 2 — E-Step (Expectation): Compute:

$$\Theta^{(t+1)} = \operatorname{argmax} Q(\theta | \theta^{(t)})$$

This means: find the **expected** complete-data log-likelihood using the current parameters.

Step 3 — M-Step (Maximization):

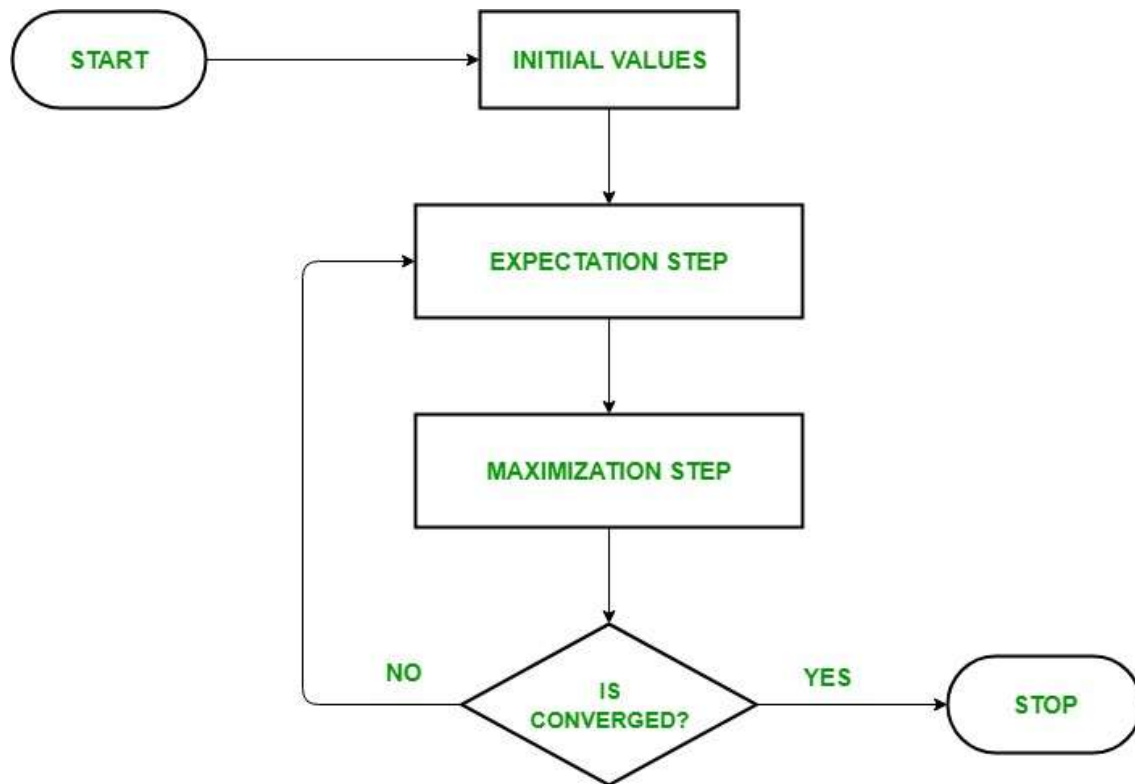
Update parameters by:

$$\theta^{(t+1)} = \operatorname{argmax} Q(\theta | \theta^{(t)})$$

This means: choose parameters that maximize the expected log-likelihood from the E-step.

Step 4 — Repeat:

Alternate E-step and M-step until parameters stop changing significantly (convergence).



HIDDEN MARKOV MODEL (HMM)

It is a statistical model that is used to describe the probabilistic relationship between a sequence of observations and a sequence of hidden states.

- An HMM consists of two types of variables: **hidden states and observations**.
 - The **hidden states** are the underlying variables that generate the observed data, but they are not directly observable.
 - The **observations** are the variables that are measured and observed.
- Markov Model is widely used in *Speech Recognition, Natural Language Modelling, On-line Handwriting Recognition, Analysis of Biological sequences such as Proteins, DNA*.
- The Hidden Markov Model (HMM) is the relationship between the hidden states and the observations using two sets of probabilities: the transition probabilities and the emission probabilities.
 - The **transition probabilities** describe the probability of transitioning from one hidden state to another.
 - The **emission probabilities** describe the probability of observing an output given a hidden state.

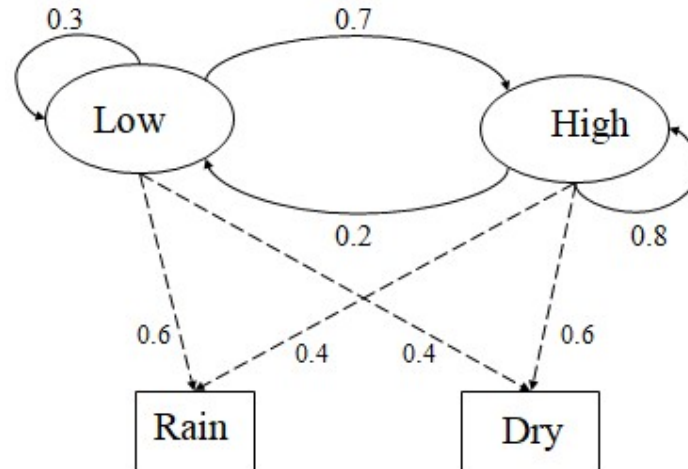
Hidden Markov Model Algorithm

- **Set of hidden states:** $\{s_1, s_2, \dots, s_N\}$
- **Process moves from one state to another generating a sequence of states:**

$$s_{i1}, s_{i2}, \dots, s_{ik}, \dots$$

- **Set of Observations:** $\{o_1, o_2, \dots, o_k\}$
- **Markov chain property:** probability of each subsequent state depends only on what was the previous state: $P(s_{ik} | s_{i1}, s_{i2}, \dots, s_{ik-1}) = P(s_{ik} | s_{ik-1})$
- To define Markov model, the following probabilities have to be specified:
 - **transition probabilities** $a_{ij} = P(s_i | s_j)$
 - **initial probabilities** $\pi_i = P(s_i)$
 - **emission probabilities** $P(o_i | s_i)$

Example of Hidden Markov Model:



- **Two states:** 'Low' and 'High' atmospheric pressure.
- **Two observations:** 'Rain' and 'Dry'.
- **Transition probabilities:**
 - $P(\text{'Low'} | \text{'Low'}) = 0.3$
 - $P(\text{'High'} | \text{'Low'}) = 0.7$
 - $P(\text{'Low'} | \text{'High'}) = 0.2$
 - $P(\text{'High'} | \text{'High'}) = 0.8$
- **Emission/Observation probabilities:**
 - $P(\text{'Rain'} | \text{'Low'}) = 0.6$
 - $P(\text{'Dry'} | \text{'Low'}) = 0.4$
 - $P(\text{'Rain'} | \text{'High'}) = 0.4$
 - $P(\text{'Dry'} | \text{'High'}) = 0.6$
- **Initial probabilities:** say
 - $P(\text{'Low'}) = 0.4$
 - $P(\text{'High'}) = 0.6$

Calculation of observation sequence probability:

- Suppose we want to calculate a probability of a sequence of observations in our example, $\{\text{'Dry'}, \text{'Rain'}\}$.
- Consider all possible hidden state sequences:

$$P(\{\text{'Dry'}, \text{'Rain'}\}) = P(\{\text{'Dry'}, \text{'Rain'}\}, \{\text{'Low'}, \text{'Low'}\}) + P(\{\text{'Dry'}, \text{'Rain'}\}, \{\text{'Low'}, \text{'High'}\}) + P(\{\text{'Dry'}, \text{'Rain'}\}, \{\text{'High'}, \text{'Low'}\}) + P(\{\text{'Dry'}, \text{'Rain'}\}, \{\text{'High'}, \text{'High'}\})$$

where first term is :

$$P(\{\text{'Dry'}, \text{'Rain'}\}, \{\text{'Low'}, \text{'Low'}\}) = P(\{\text{'Dry'}, \text{'Rain'}\} | \{\text{'Low'}, \text{'Low'}\}) P(\{\text{'Low'}, \text{'Low'}\})$$

$$\begin{aligned}
&= P(\text{'Dry'}|\text{'Low'})P(\text{'Rain'}|\text{'Low'})P(\text{'Low'})P(\text{'Low'}|\text{'Low'}) \\
&= 0.4*0.6*0.4*0.3
\end{aligned}$$

Similary, Calculate others

Select maximum probability from among them.

If we had a set of states, we could calculate the probability of the sequence. But we don't have the states. All we have are a sequence of observations.

Problem Associated with HMM model:

- **Problem 1 (Evaluation):** Given an HMM and a sequence of observations, what is the probability that the observations are generated by the model? *Forward-Backward algorithm solve this problem.*
- **Problem 2 (Decoding):** Given an HMM and a sequence of observations, what is the most likely state sequence in the model that produced the observations? *Viterbi algorithm solve this problem.*
- **Problem 3 (Learning):** Given an observation sequence, how should the model parameters be adjusted to maximize the probability of the evaluation problem? *Baum-Welch algorithm solve this problem*

■ Algorithm 2. (HMM Forward)

```

1 initialize  $t \leftarrow 0, a_{ij}, b_{jk}, \text{visible sequence } \mathbf{V}^T, \alpha_j(0)$ 
2 for  $t \leftarrow t + 1$ 
3    $\alpha_j(t) \leftarrow b_{jk}v(t) \sum_{i=1}^c \alpha_i(t-1)a_{ij}$ 
4 until  $t = T$ 
5 return  $P(\mathbf{V}^T) \leftarrow \alpha_0(T)$  for the final state
6 end

```

■ Algorithm 3. (HMM Backward)

```

1 initialize  $\beta_j(T), t \leftarrow T, a_{ij}, b_{jk}, \text{visible sequence } \mathbf{V}^T$ 
2 for  $t \leftarrow t - 1$ ;
3    $\beta_i(t) \leftarrow \sum_{j=1}^c \beta_j(t+1)a_{ij}b_{jk}v(t+1)$ 
4 until  $t = 1$ 
5 return  $P(\mathbf{V}^T) \leftarrow \beta_i(0)$  for the known initial state
6 end

```

■ Algorithm 4. (HMM decoding)

```

1 begin initialize Path  $\leftarrow \{\}, t \leftarrow 0$ 
2   for  $t \leftarrow t + 1$ 
3      $j \leftarrow j + 1$ 
4     for  $j \leftarrow j + 1$ 
5        $\alpha_j(t) \leftarrow b_{jk}v(t) \sum_{i=1}^c \alpha_i(t-1)a_{ij}$ 
6     until  $j = c$ 
7      $j' \leftarrow \arg \max_j \alpha_j(t)$ 
8     Append  $\omega_{j'}$  to Path
9   until  $t = T$ 
10 return Path
11 end

```
